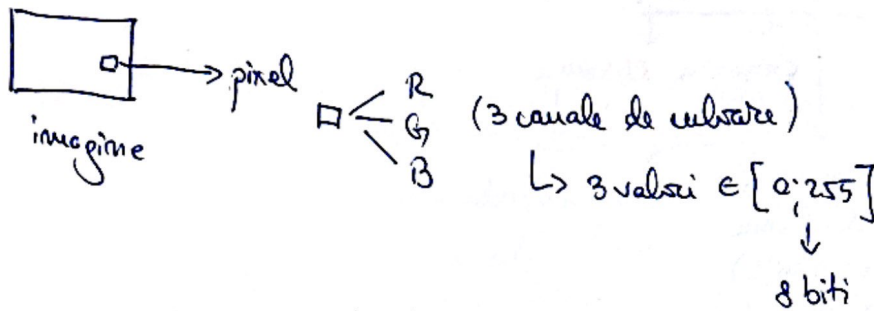


Discrete Cosine Transform /

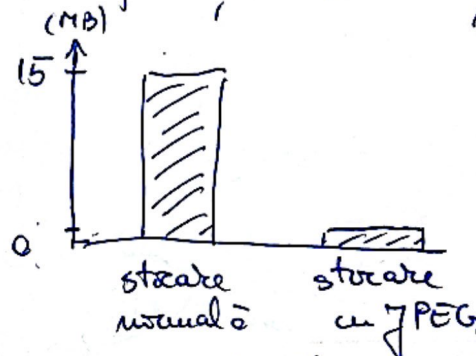
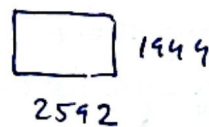
JPEG

YT: The Unreasonable
Effectiveness of JPEG!
A Signal Processing
Approach



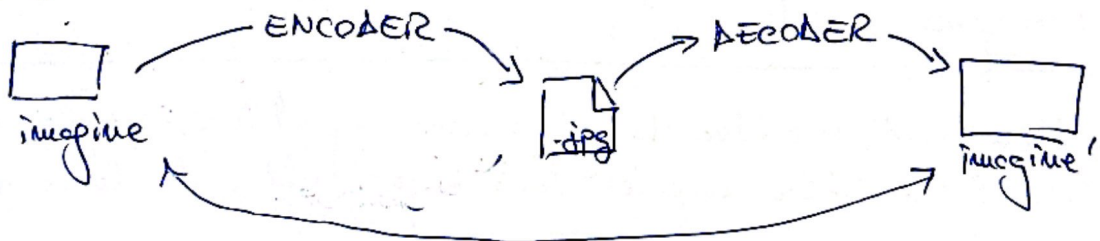
$\Rightarrow$ O imagine stocată normal folosește $3 \cdot 8' = 24 \text{ biti} = 3 \text{ bytes/pixel}$

Ex:



Jpeg vrea să reprezinte imaginea fol. cât mai puțină memorie și
minimizând efectele colaterale asupra calității acesteia!

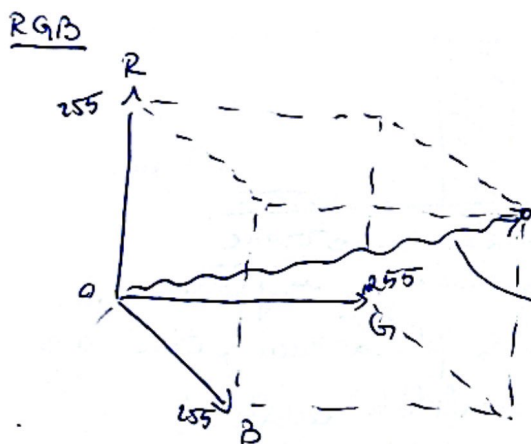
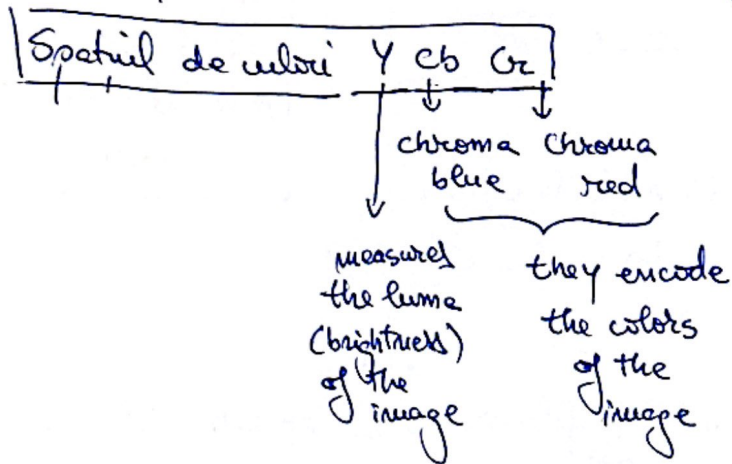
Jpeg folosește LOSSY COMPRESSION



in the compression part of the pipeline we are going
to deliberately lose information

$\Rightarrow$ What information can we get rid of and how?

- Our eyes are more sensitive to brightness than color.



As you progress on the diagonal from the origin to the color (255, 255, 255), you get gradually brighter colors.

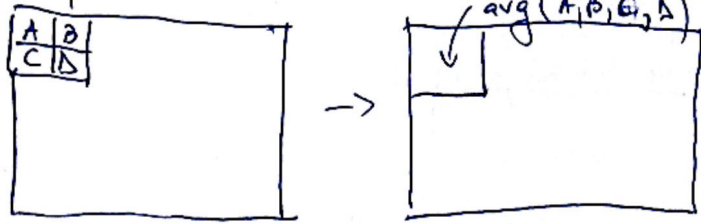
This line defines all possible grayscale colors, which are a direct measure for brightness.

The idea of separating brightness from color is core to the YCbCr color space.

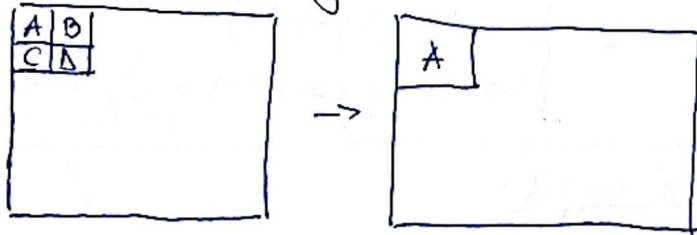
- As a result, one idea to compress our original image involves sampling less of the CbCr components and keeping all of the luma components.

|| CHROMA DOWN SAMPLING (SUBSAMPLING) ||

→ One approach that defines a 4-2-0 chroma downsampling scheme:



→ Chroma subsampling 4-2-0 :

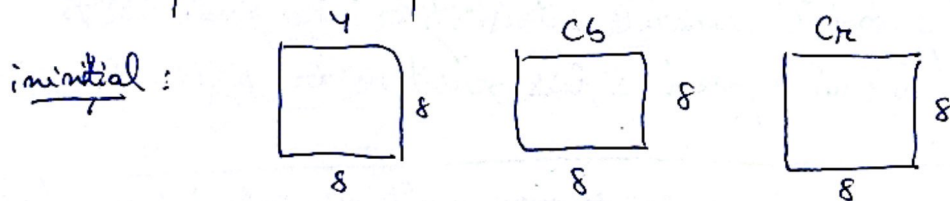


top left pixel is the color of the entire 2x2 block

→ In real world images is often hard to see any changes after subsampling.

By merging 2x2 blocks on the Cb and Cr channels into color, we are left with $\frac{1}{4}$ of the original data in each color channel, shrinking the total file size by 50%.

→ Pt un patch de 8x8 pixels:



pixel count:

	Y	Cb	Cr
after subsampling	64/64	16/64	16/64
pixel ratio :	100 %	25 %	25 %

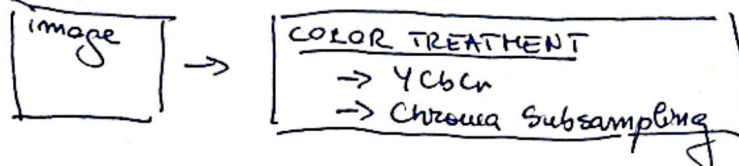
fraction of the image :

	Y	Cb	Cr
	$\frac{1}{3}$	$\frac{1}{12}$	$\frac{1}{12}$

Final total size compared to original :

$\frac{6}{12} = 50 \% .$

Review



Next

JPEG ENCODER

↳ Forward DCT

↳ Quantization

↳ Lossless Encoder

→ with focus on the Y channel (which essentially defines grayscale)

→ same principles apply to CbCr

Frequency components in images

High frequency components correspond to rapid changes between pixels. Lower frequency components are related to smoother changes.

high frequency



low frequency

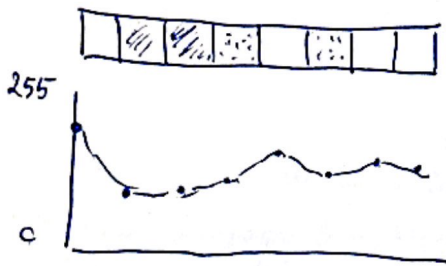


- ① ∇ Real world images tend to have more low frequency components.
○ (Dacă selectez random secvențe de pixeli dintr-o imagine, cel mai probabil variabile intensității lor între pixeli vor fi mici.)
- ② ∇ Human visual system is less sensitive to higher frequency detail.
○

⇒ Pot comprima imaginile și mai mult diminuând frecvențele înalte din ea.

Cum obțin componentele de frecvență ale unei imagini? Cu DCT.

DCT

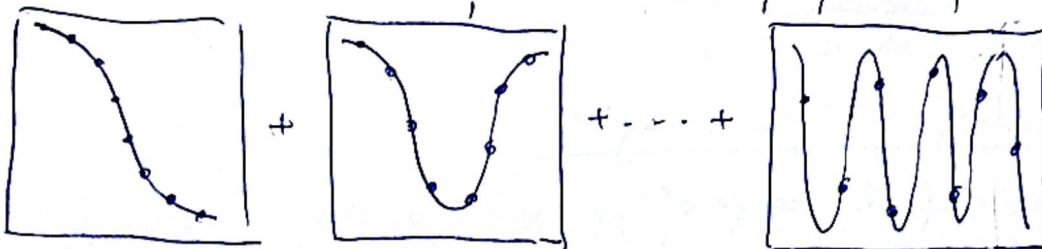


pixeli

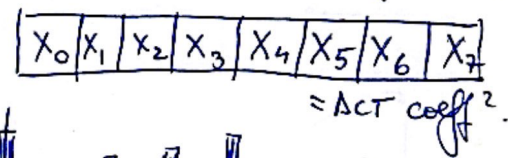
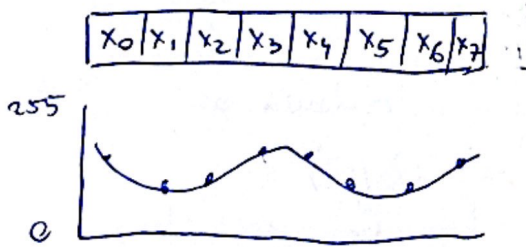
esantioame ale unui semnal

pe care îl reprezentăm ca $\sum \cosinusuri$.

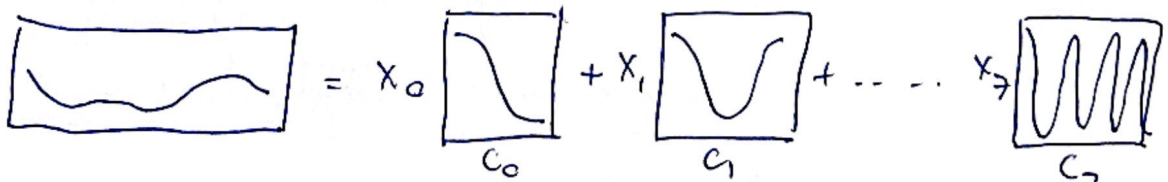
un esantion = $\sum$ sample points from cosine waves



DCT = takes an input of sample points from our original signal and gives an output of the same size. (= weights of cosine waves of different frequencies, that contribute to the original signal)

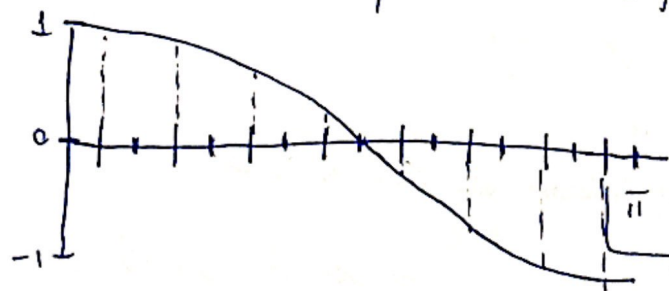


= DCT coeff².



// unraveling a complex signal into a weighted sum of simpler cosine waves.

→ How should I sample the cosine function?



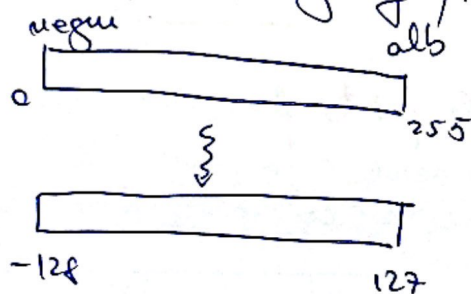
pts & samples are: $\pi/2N$ & segments segments

$$\Rightarrow y_m = \cos \frac{(2m+1)\pi}{2N}$$

$\downarrow$
 ex: $m=0$

Pls apply ΔC to image:

→ we shift the range of pixel values by 128



shifting the signal up or down only affects the index $x[0]$

measure of the overall brightness of a set of pixels.
brighter images → larger $x[0]$

$$X[k] = \sum_{m=0}^{N-1} x_m \cdot \cos \frac{(2m+1)k\pi}{2N}$$

$\downarrow$
 sample dir - cosine wave
 (incorporates the frequency of the cosine wave)

$$X_Q = \sum_{n=0}^{N-1} x_n \cdot \cos \frac{(2n+1)k\pi}{2N} =$$

$$= \begin{bmatrix} x_0 & x_1 & \dots & x_{N-1} \end{bmatrix} \cdot \underbrace{\begin{bmatrix} \cos \frac{k\pi}{2N} \\ \cos \frac{3k\pi}{2N} \\ \vdots \\ \cos \frac{(2N-1)k\pi}{2N} \end{bmatrix}}_{C_k} =$$

$$= \vec{x}^t \cdot \vec{C}_k =$$

$$= \vec{C}_k^t \cdot \vec{x}$$

$$\Rightarrow \begin{bmatrix} x_0 \\ \vdots \\ x_{N-1} \end{bmatrix} = \underbrace{\begin{bmatrix} \leftarrow C_0^t \rightarrow \\ \leftarrow C_1^t \rightarrow \\ \vdots \\ \leftarrow C_{N-1}^t \rightarrow \end{bmatrix}}_M \begin{bmatrix} x_0 \\ \vdots \\ x_{N-1} \end{bmatrix}$$

all row vectors in this matrix are orthogonal to each other, i.e. $\vec{C}_i^t \cdot \vec{C}_j = 0, \forall i \neq j$

$$\begin{cases} \vec{X} = M \cdot \vec{x} \\ \vec{x} = M^{-1} \vec{X} \end{cases}$$

here $M^{-1} = M^t$ * with additional normalization

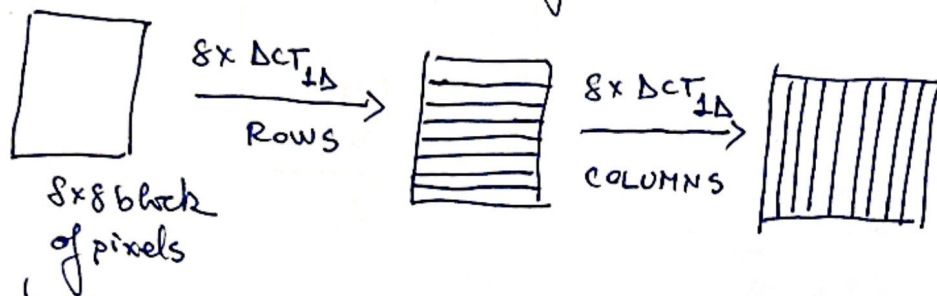
Inverse DCT

$$\begin{bmatrix} x_0 \\ \vdots \\ x_{N-1} \end{bmatrix} = x_0 \begin{bmatrix} \uparrow \\ c_0 \\ \downarrow \end{bmatrix} + x_1 \begin{bmatrix} \uparrow \\ c_1 \\ \downarrow \end{bmatrix} + \dots + x_{N-1} \begin{bmatrix} \uparrow \\ c_{N-1} \\ \downarrow \end{bmatrix}$$

⑦

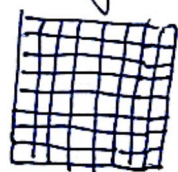
JPEG

- takes an image and splits it into 8×8 blocks
- centers their values around 0 by subtracting 128
- we take the block and apply ΔCT_{1D} to each row of the block, giving us 8 sets of ΔCT coefficients



$\Rightarrow DCT_{2D}$

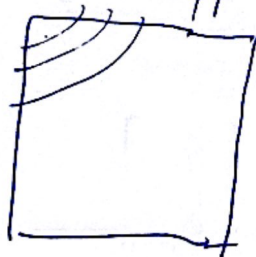
$\Rightarrow$ 64 coefficients each of which are a weight on a specific 8×8 pattern.



$\downarrow$ 1st row
1st col. $\rightarrow$ correspond to the 1D patterns
the other elements = compositions of the 1D patterns $\rightarrow x$

∇ We can build up any 8×8 image using these 64 fundamental patterns.

∇ Most of the largest values of the ΔCT coefficients are in the upper left corner, which corresponds to lower frequency components.



2D ΔCT coeff²

ENERGY COMPACTION

- the process of eliminating higher frequency components = QUANTIZATION

↳ Given an 8×8 matrix of frequency coefficients from the DCT

↳ divide each element by a scalar value

↳ and round it to an integer.

in the
ENCODING
stage

in the DECODING stage: we multiply this matrix by the same quantization matrix. elem by elem
 $\Rightarrow$ a matrix different from the original one
 $\Rightarrow$ lost info, BUT most of the lower frequency components will be retained

- dupo importante elem cu elem la matricea de cuantizare $\Rightarrow$
 $\Rightarrow$ multe zerouri in dreptunghi.

- In practice: different quantization tables for $\begin{pmatrix} Y \\ CbCr \end{pmatrix}$

here quantization can be even more aggressive

- dupo cuantizare:

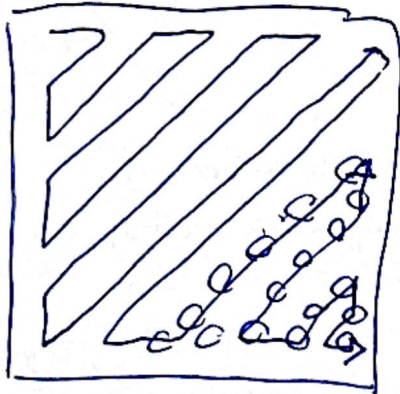
21	13	-1	1	0	0	0	0
14	-3						0
-1	4						0
3							0
							0
							0
							0
0	0	0	0	0	0	0	0

8

exploit REDUNDANCY!

- LOSSLESS ENCODER $\rightarrow$ involves a combination of run length encoding
 Huffman encoding

- A jpeg encoder will order the coefficients in a zigzag manner to maximize the chance of a large sequence of zeros in order



→ $\{90, -40, 0, 5, 0[x2], 4, 0[x3], 11, 0[x8], -3, 0[x44]\}$

↓ JPEG, Specific Run-length Encoding

$[(r, s), c]$

→ r = number of 0's preceding value

→ s = number of bits needed to encode value r

→ c = coefficient value

→ $(0, 0)$ indicates the end of block (all 0's)

⇒ $\{[(0, 7), 90], [(0, 6), -40], [(1, 3), 5], \dots, [(0, 0)]\}$

